

**Laboratory work 1:
Study and Empirical Analysis of Algorithms for
Determining
Fibonacci N-th Term**

Elaborated:
st. gr. FAF-233

Iacovlev Maxim

Verified:
asist. univ.

Fiștic Cristofor

TABLE OF CONTENTS

ALGORITHM ANALYSIS	3
Objective	3
Tasks.....	3
Theoretical Notes:	3
Introduction:	4
Comparison Metric:	4
Input Format:.....	4
IMPLEMENTATION.....	5
Recursive Method:	5
Dynamic Programming Method:.....	6
Matrix Power Method:	8
Binet Formula Method:	11
Fast Matrix Multiplication Method:	12
Fast Double Method:	14
Memoization	15
CONCLUSION	17

ALGORITHM ANALYSIS

Objective

Study and analyze different algorithms for determining Fibonacci n -th term.

Tasks:

1. Implement at least 3 algorithms for determining Fibonacci n -th term;
2. Decide properties of input format that will be used for algorithm analysis;
3. Decide the comparison metric for the algorithms;
4. Analyze empirically the algorithms;
5. Present the results of the obtained data;
6. Deduce conclusions of the laboratory.

Theoretical Notes:

An alternative to mathematical analysis of complexity is empirical analysis.

This may be useful for: obtaining preliminary information on the complexity class of an algorithm; comparing the efficiency of two (or more) algorithms for solving the same problems; comparing the efficiency of several implementations of the same algorithm; obtaining information on the efficiency of implementing an algorithm on a particular computer.

In the empirical analysis of an algorithm, the following steps are usually followed:

1. The purpose of the analysis is established.
2. Choose the efficiency metric to be used (number of executions of an operation (s) or time execution of all or part of the algorithm).
3. The properties of the input data in relation to which the analysis is performed are established (data size or specific properties).
4. The algorithm is implemented in a programming language.
5. Generating multiple sets of input data.
6. Run the program for each input data set.
7. The obtained data are analyzed.

The choice of the efficiency measure depends on the purpose of the analysis. If, for example, the aim is to obtain information on the complexity class or even checking the accuracy of a theoretical estimate then it is appropriate to use the number of operations performed. But if the goal is to assess the behavior of the implementation of an algorithm then execution time is appropriate.

After the execution of the program with the test data, the results are recorded and, for the purpose of the analysis, either synthetic quantities (mean, standard deviation, etc.) are calculated or a graph with appropriate pairs of points (i.e. problem size, efficiency measure) is plotted.

Introduction:

The Fibonacci sequence is the series of numbers where each number is the sum of the two preceding numbers. For example: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, ... Mathematically we can describe this as: $x_n = x_{n-1} + x_{n-2}$.

Many sources claim this sequence was first discovered or "invented" by Leonardo Fibonacci. The Italian mathematician, who was born around A.D. 1170, was initially known as Leonardo of Pisa. In the 19th century, historians came up with the nickname Fibonacci (roughly meaning "son of the Bonacci clan") to distinguish the mathematician from another famous Leonardo of Pisa.

There are others who say he did not. Keith Devlin, the author of Finding Fibonacci: The Quest to Rediscover the Forgotten Mathematical Genius Who Changed the World, says there are ancient Sanskrit texts that use the Hindu-Arabic numeral system - predating Leonardo of Pisa by centuries.

But, in 1202 Leonardo of Pisa published a mathematical text, Liber Abaci. It was a "cookbook" written for tradespeople on how to do calculations. The text laid out the Hindu-Arabic arithmetic useful for tracking profits, losses, remaining loan balances, etc, introducing the Fibonacci sequence to the Western world.

Traditionally, the sequence was determined just by adding two predecessors to obtain a new number, however, with the evolution of computer science and algorithmics, several distinct methods for determination have been uncovered. The methods can be grouped in 4 categories, Recursive Methods, Dynamic Programming Methods, Matrix Power Methods, and Benet Formula Methods. All those can be implemented naively or with a certain degree of optimization, that boosts their performance during analysis.

As mentioned previously, the performance of an algorithm can be analyzed mathematically (derived through mathematical reasoning) or empirically (based on experimental observations).

Within this laboratory, we will be analyzing the 4 naïve algorithms empirically.

Comparison Metric:

The comparison metric for this laboratory work will be considered the time of execution of each algorithm ($T(n)$)

Input Format:

As input, each algorithm will receive two series of numbers that will contain the order of the Fibonacci terms being looked up. The first series will have a more limited scope, (5, 7, 10, 12, 15, 17, 20, 22, 25, 27, 30, 32, 35, 37, 40, 42, 45), to accommodate the recursive method, while the second series will have a bigger scope to be able to compare the other algorithms between themselves (501, 631, 794, 1000, 1259, 1585, 1995, 2512, 3162, 3981, 5012, 6310, 7943, 10000, 12589, 15849).

IMPLEMENTATION

All four algorithms will be implemented in their naïve form in python and analyzed empirically based on the time required for their completion. While the general trend of the results may be similar to other experimental observations, the particular efficiency in rapport with input will vary depending on memory of the device used.

The error margin determined will constitute 2.5 seconds as per experimental measurement.

Recursive Method:

The recursive method, also considered the most inefficient method, follows a straightforward approach of computing the n -th term by computing its predecessors first, and then adding them. However, the method does it by calling upon itself a number of times and repeating the same operation, for the same term, at least twice, occupying additional memory and, in theory, doubling its execution time.

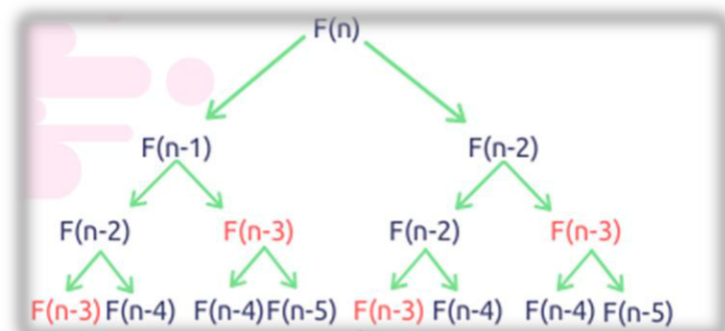


Figure 1 Fibonacci Recursion

Algorithm Description:

The naïve recursive Fibonacci method follows the algorithm as shown in the next pseudocode:

Fibonacci(n) :

 if $n \leq 1$:

 return n

 otherwise:

 return Fibonacci($n-1$) + Fibonacci($n-2$)

Implementation:

```
def fibonacci(x):  
    if x <= 1:  
        return x  
    else:  
        return fibonacci(x-1)+ fibonacci(x-2)
```

Figure 2 Fibonacci recursion in Python

Results:

After running the function for each n Fibonacci term proposed in the list from the first Input Format and saving the time for each n, we obtained the following results:

	5	7	10	12	15	17	20	22	25	27	30	32	35	37	40	42	45
0	0.0	0.0	0.0	0.0	0.001	0.0011	0.0022	0.0	0.075	0.14	0.66	1.87	7.44	21.11	55.05	136.89	790.019
1	0.0	0.0	0.0	0.0	0.000	0.0000	0.0000	0.0	0.000	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.000
2	0.0	0.0	0.0	0.0	0.000	0.0000	0.0000	0.0	0.000	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.000
3	0.0	0.0	0.0	0.0	0.000	0.0000	0.0000	0.0	0.000	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.000

Figure 3 Results for first set of inputs

In Figure 3 is represented the table of results for the first set of inputs. The highest line(the name of the columns) denotes the Fibonacci n-th term for which the functions were run. Starting from the second row, we get the number of seconds that elapsed from when the function was run till when the function was executed. We may notice that the only function whose time was growing for this few n terms was the Recursive Method Fibonacci function.

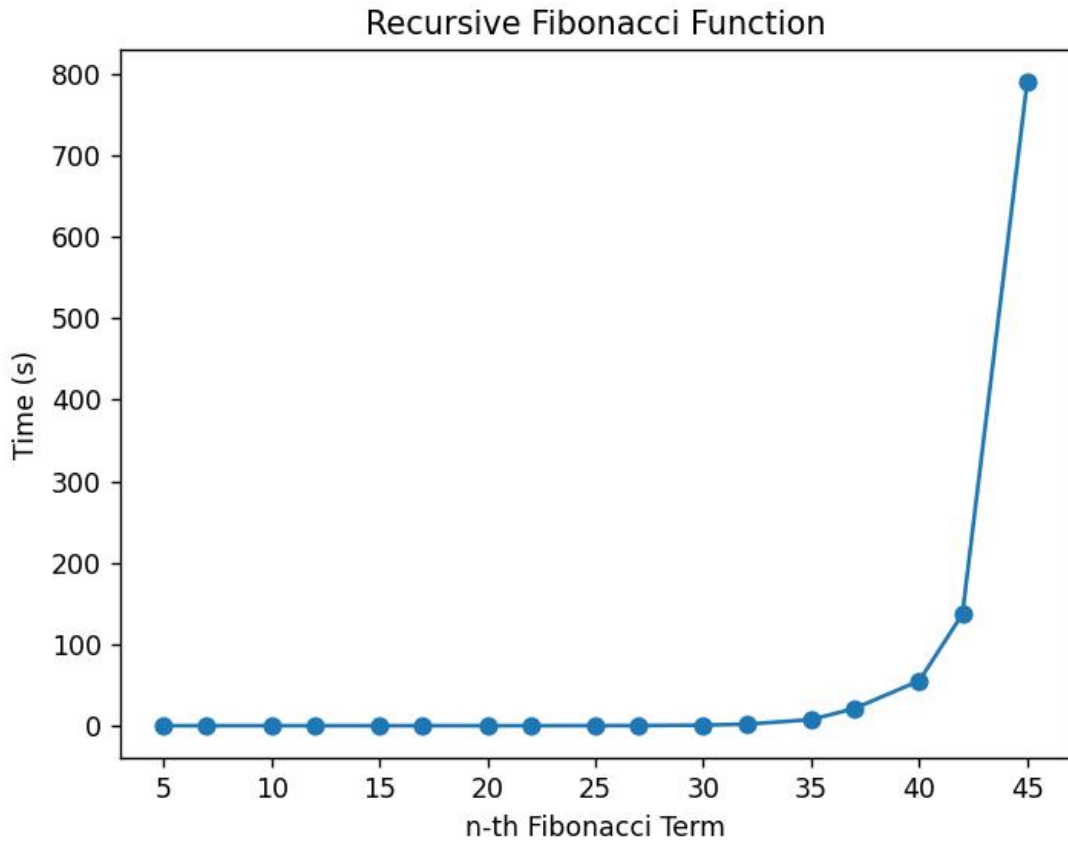


Figure 4 Graph of Recursive Fibonacci Function

Not only that, but also in the graph in Figure 4 that shows the growth of the time needed for the operations, we may easily see the spike in time complexity that happens after the 42nd term, leading us to deduce that the Time Complexity is exponential. $T(2)$.

Dynamic Programming Method:

The Dynamic Programming method, similar to the recursive method, takes the straightforward approach of calculating the n-th term. However, instead of calling the function upon itself, from top down

it operates based on an array data structure that holds the previously computed terms, eliminating the need to recompute them.

Algorithm Description:

The naïve DP algorithm for Fibonacci n-th term follows the pseudocode:

```
Fibonacci(n) :
    Array A;
    A[0] ← 0;
    A[1] ← 1;
    for i ← 2 to n - 1 do
        A[i] ← A[i-1] + A[i-2];
    return A[n-1]
```

Implementation:

```
def f(x):
    l1 = [0, 1]

    for i in range(2, x + 1):
        l1.append(l1[i-1] + l1[i-2])

    return l1[x]
```

Figure 5 Fibonacci DP in Python

Results:

After the execution of the function for each n Fibonacci term mentioned in the second set of Input Format we obtain the following results:

	501	631	794	1000	1259	1585	1995	2512	3162	3981	5012	6310	7943	10000	12589	15849
0	0.0	0.0	0.000726	0.0	0.000758	0.0007	0.000360	0.003646	0.001101	0.001459	0.001831	0.002550	0.004351	0.005489	0.019695	0.014626
1	0.0	0.0	0.000000	0.0	0.000000	0.0000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.001074	0.000000	0.000727
2	0.0	0.0	0.000000	0.0	0.000000	0.0000	0.033937	0.014543	0.013128	0.013862	0.020456	0.009831	0.025155	0.037205	0.076604	0.064923

Figure 6 Fibonacci DP Results

With the Dynamic Programming Method (first row, row[0]) showing excellent results with a time complexity denoted in a corresponding graph of T(n),

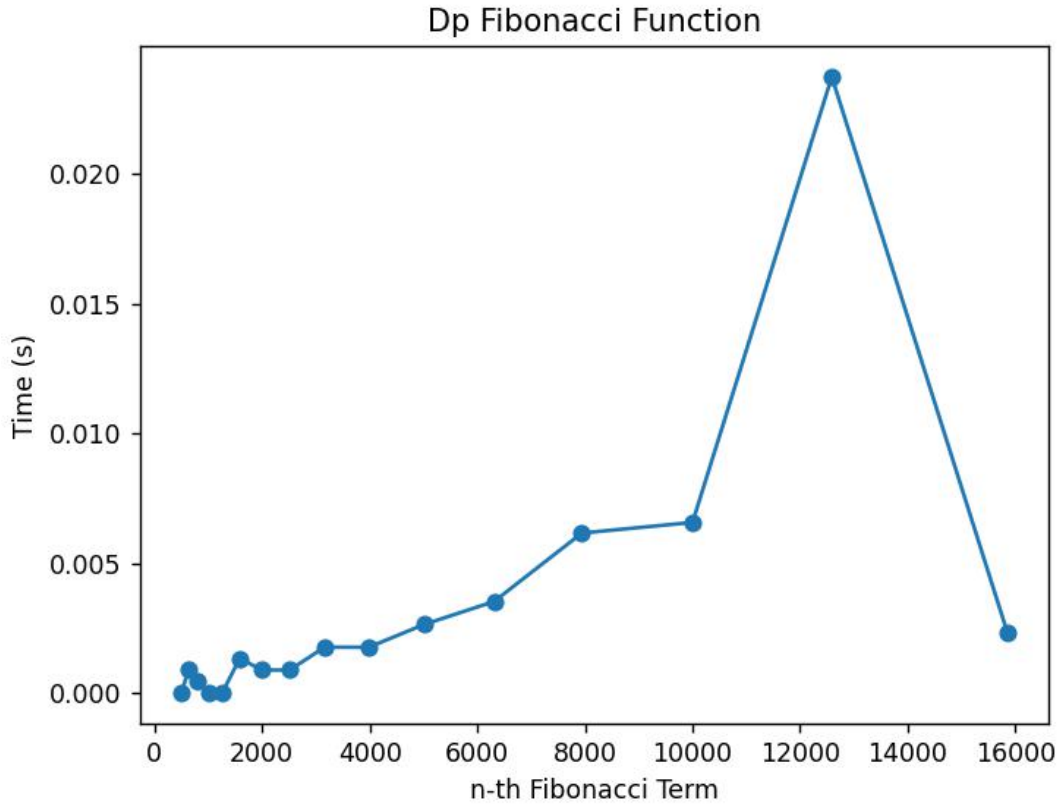


Figure 7 Fibonacci DP Graph

Matrix Power Method:

The Matrix Power method of determining the n-th Fibonacci number is based on, as expected, the multiple multiplication of a naïve Matrix $\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$ with itself

Algorithm Description:

It is known that

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} b \\ a + b \end{pmatrix}$$

This property of Matrix multiplication can be used to represent

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} F_1 \\ F_2 \end{pmatrix}$$

And similarly:

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} F_1 \\ F_2 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^2 \begin{pmatrix} F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} F_2 \\ F_3 \end{pmatrix}$$

Which turns into a general

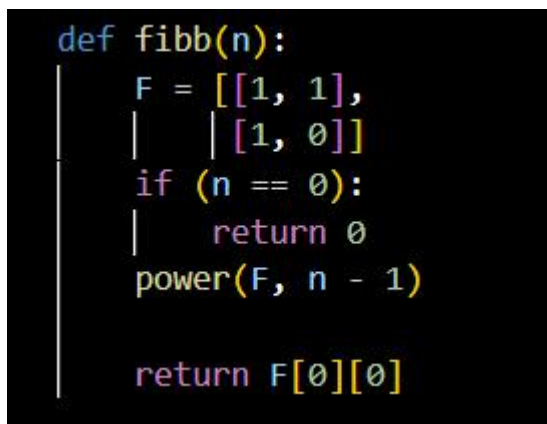
$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^n \begin{pmatrix} F_0 \\ F_1 \end{pmatrix} = \begin{pmatrix} F_n \\ F_{n+1} \end{pmatrix}$$

This set of operation can be described in pseudocode as follows:

```
Fibonacci(n):  
    F <- []  
    vec <- [[0], [1]]  
    Matrix <- [[0, 1], [1, 1]]  
    F <- power(Matrix, n)  
    F <- F * vec  
    Return F[0][0]
```

Implementation:

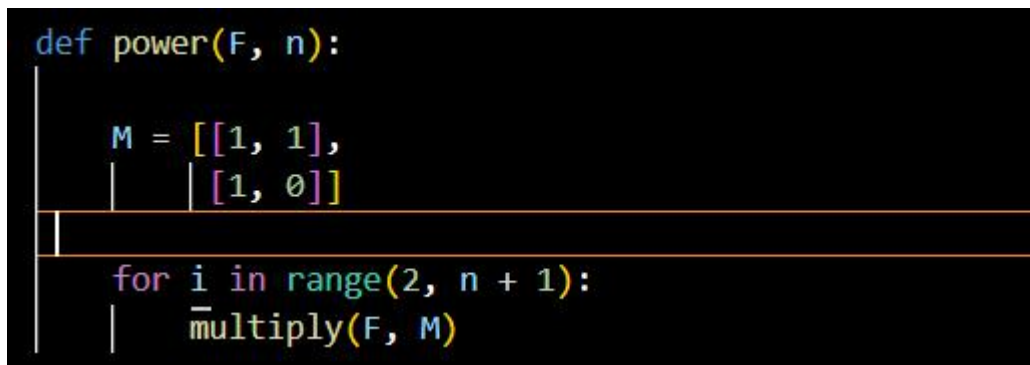
The implementation of the driving function in Python is as follows:

A screenshot of a code editor with a dark background. The code defines a function 'fibb(n)'. It initializes a matrix 'F' as [[1, 1], [1, 0]]. It has a base case 'if (n == 0): return 0'. For other values of 'n', it calls 'power(F, n - 1)' and then returns 'F[0][0]'.

```
def fibb(n):  
    F = [[1, 1],  
         | [1, 0]]  
    if (n == 0):  
        return 0  
    power(F, n - 1)  
  
    return F[0][0]
```

Figure 8 Fibonacci Matrix Power Method in Python

With additional miscellaneous functions:

A screenshot of a code editor with a dark background. The code defines a function 'power(F, n)'. It initializes a matrix 'M' as [[1, 1], [1, 0]]. It then enters a loop 'for i in range(2, n + 1):' where it calls 'multiply(F, M)'.

```
def power(F, n):  
    M = [[1, 1],  
         | [1, 0]]  
    for i in range(2, n + 1):  
        multiply(F, M)
```

Figure 9 Power Function Python

Where the power function (Figure 8) handles the part of raising the Matrix to the power n, while the multiplying function (Figure 9) handles the matrix multiplication with itself.

```
def multiply(F, M):
    x = (F[0][0] * M[0][0] +
         F[0][1] * M[1][0])
    y = (F[0][0] * M[0][1] +
         F[0][1] * M[1][1])
    z = (F[1][0] * M[0][0] +
         F[1][1] * M[1][0])
    w = (F[1][0] * M[0][1] +
         F[1][1] * M[1][1])

    F[0][0] = x
    F[0][1] = y
    F[1][0] = z
    F[1][1] = w
```

Figure 10 Multiply Function Python

Results:

After the execution of the function for each n Fibonacci term mentioned in the second set of Input Format we obtain the following results:

	501	631	794	1000	1259	1585	1995	2512	3162	3981	5012	6310	7943	10000	12589	15849
0	0.0	0.0	0.000726	0.0	0.000758	0.0007	0.000360	0.003646	0.001101	0.001459	0.001831	0.002550	0.004351	0.005489	0.019695	0.014626
1	0.0	0.0	0.000000	0.0	0.000000	0.0000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.001074	0.000000	0.000727
2	0.0	0.0	0.000000	0.0	0.000000	0.0000	0.033937	0.014543	0.013128	0.013862	0.020456	0.009831	0.025155	0.037205	0.076604	0.064923

Figure 11 Matrix Method Fibonacci Results

With the naïve Matrix method (indicated in last row, row[2]), although being slower than the Binet and Dynamic Programming one, still performing pretty well, with the form of the graph indicating a pretty solid $T(n)$ time complexity.

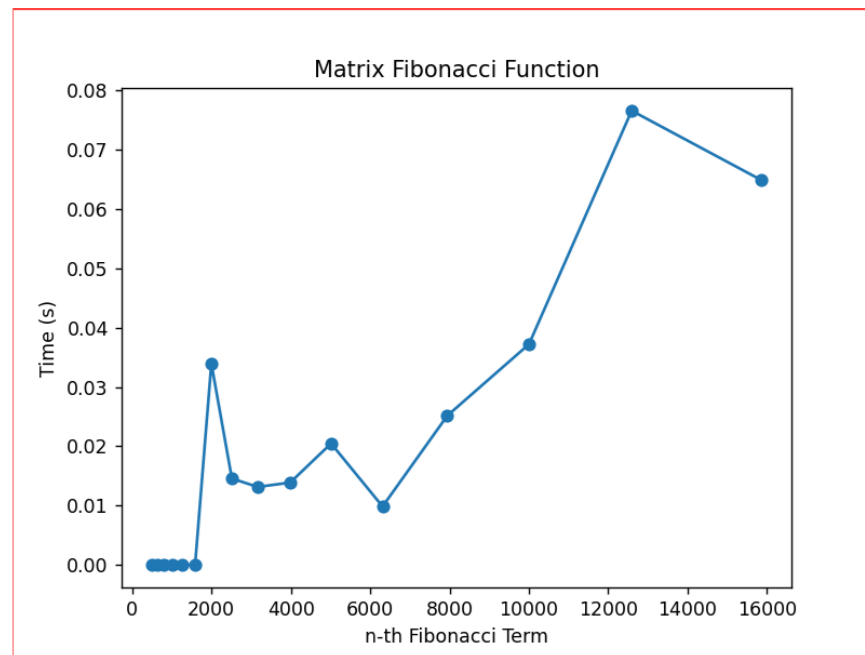


Figure 12 Matrix Method Fibonacci graph

Binet Formula Method:

The Binet Formula Method is another unconventional way of calculating the n-th term of the Fibonacci series, as it operates using the Golden Ratio formula, or phi. However, due to its nature of requiring the usage of decimal numbers, at some point, the rounding error of python that accumulates, begins affecting the results significantly. The observation of error starting with around 70-th number making it unusable in practice, despite its speed.

Algorithm Description:

The set of operation for the Binet Formula Method can be described in pseudocode as follows:

Fibonacci(n) :

```
phi <- (1 + sqrt(5))
phi1 <- (1 - sqrt(5))
return pow(phi, n) - pow(phi1, n) / (pow(2, n) * sqrt(5))
```

Implementation:

The implementation of the function in Python is as follows, with some alterations that would increase the number of terms that could be obtain through it:

```
def fib(x):
    ctx = Context(prec=60, rounding=ROUND_HALF_EVEN)
    phi = Decimal((1 + Decimal(5**(1/2))))
    phi2 = Decimal((1 - Decimal(5**(1/2))))

    return int((ctx.power(phi, Decimal(x)) - ctx.power(phi2, Decimal(x))) / (2**x * Decimal(5**(1/2))))
```

Figure 13 Fibonacci Binet Formula Method in Python

Results:

Although the most performant with its time, as shown in the table of results, in row [1],

	501	631	794	1000	1259	1585	1995	2512	3162	3981	5012	6310	7943	10000	12589	15849
0	0.0	0.0	0.000726	0.0	0.000758	0.0007	0.000360	0.003646	0.001101	0.001459	0.001831	0.002550	0.004351	0.005489	0.019695	0.014626
1	0.0	0.0	0.000000	0.0	0.000000	0.0000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.001074	0.000000	0.000727
2	0.0	0.0	0.000000	0.0	0.000000	0.0000	0.033937	0.014543	0.013128	0.013862	0.020456	0.009831	0.025155	0.037205	0.076604	0.064923

Figure 14 Fibonacci Binet Formula Method results

And as shown in its performance graph,

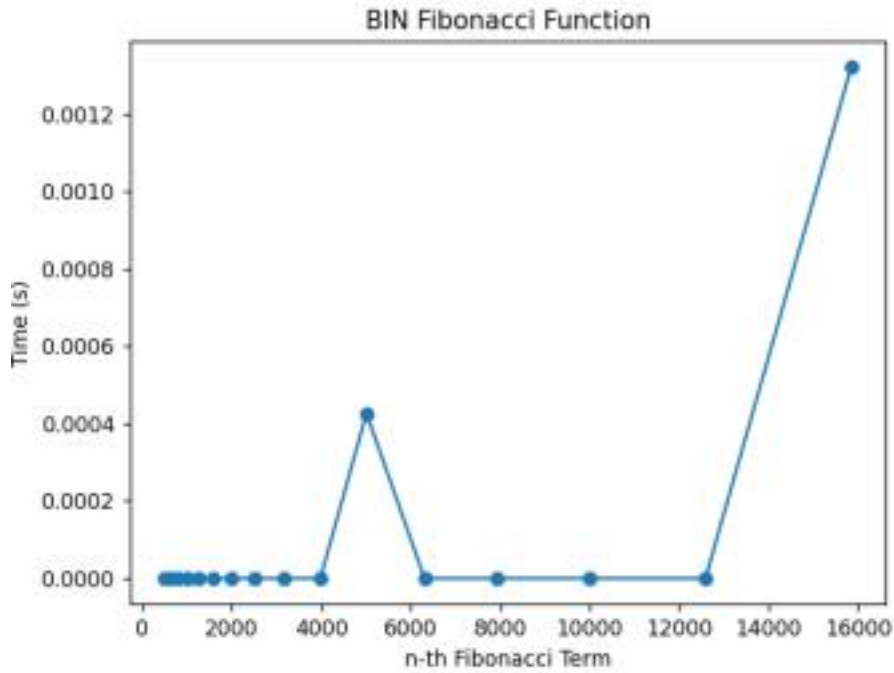


Figure 15 Fibonacci Binet formula Method

The Binet Formula Function is not accurate enough to be considered within the analysed limits and is recommended to be used for Fibonacci terms up to 80. At least in its naïve form in python, as further modification and change of language may extend its usability further.

Fast Matrix Power Method:

This method is a modification or extension of the aforementioned Matrix Power method.

Instead of raising the matrix one power at a time, it is possible to utilize the powers of 2 to reach the desired power much faster. To reach a desired power using the powers of 2, it is enough to convert the number into its binary representation and then multiplying the components, each being the initial matrix to some multiple of 2.

For example, to compute F_{21} , the fast matrix power method would imply to calculate

$$F_{21} = F_{16}F_4F_1$$

Algorithm Description:

The set of operation for the Fast Matrix Power Method can be described in pseudocode as follows:

```
Fibonacci_FastExp(n) :
    step = M2x2(0, 1, 1, 1)
    fib = M2x2(0, 1, 1, 1)
    While n > 0:
        if n is odd:
            fib <- fib * step
            step <- step * step
            n <- n // 2
    return fib[0][0]
```

Implementation:

Below is the Python implementation of the Fast Matrix Power method. I omitted the definition of the M2x2 class, which is just a helper class with override multiplication and initialisation to make the main function cleaner.

```
def fibonacci_fastexp(n):
    step = M2x2(e00: 0, e01: 1, e10: 1, e11: 1)
    fib = M2x2(e00: 0, e01: 1, e10: 1, e11: 1)

    while n > 0:
        if n & 1:
            fib *= step
        step *= step
        n >>= 1

    return fib.e00
```

Figure 16 Fibonacci Fast Matrix Multiplication Method

Results:

Below are the times of execution of the Fast Matrix Multiplication method in milliseconds. So far, this method has shown the best performance

Fast exp																	
Index:	501	631	794	1000	1259	1585	1995	2512	3162	3981	5012	6310	7943	10000	12589	15849	
Time (ms):	0.0250	0.0222	0.0219	0.0231	0.0298	0.0300	0.0370	0.0501	0.0570	0.0739	0.1352	0.1647	0.1941	0.3972	0.4568	0.5672	

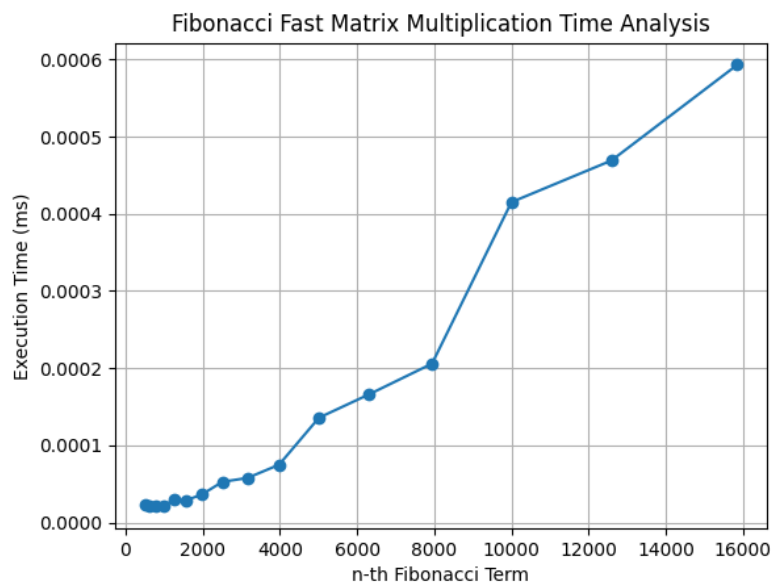


Figure 17 Fibonacci Fast Matrix Multiplication Time Analysis

Fast Matrix Multiplication method does not sacrifice any precision to achieve maximum speed. Since right-shifting is used on every iteration to reach the result, n is divided by 2 on each step, so time complexity of this algorithm is $O(\log n)$. When it comes to space complexity, no new variables are created inside the loop, so it remains constant for any desired fibonacci number n , resulting in space complexity $O(1)$.

Fast Doubling Method:

Fast Doubling Method is an efficient recursive approach to find Fibonacci numbers.

$$F(2k) = F(k) \cdot (2F(k+1) - F(k))$$

$$F(2k+1) = F(k+1)^2 + F(k)^2$$

Similarly to Fast Matrix Multiplication method, we try to reach the desired number as fast as possible by achieving the $2k$ -th fibonacci number directly from k -th one.

Algorithm Description:

The set of operation for the Fast Doubling Method can be described in pseudocode as follows:

```
Fibonacci_FastDouble(n):
    Function fib_pair(k):
        If k ≤ 1:
            Return (k, 1)
        (Fh, Fh1) <- fib_pair(k // 2)
        Fk <- Fh * (2 * Fh1 - Fh)
        Fk1 <- Fh1^2 + Fh^2
        If k is odd:
            Return (Fk1, Fk + Fk1)
        Else:
            Return (Fk, Fk1)
    Return fib_pair(n).first
```

Implementation:

Implementation is based on recursion and bitwise operations, which make it faster.

```
def fibonacci_fast_double(n: int) -> int:
    def fib_pair(k: int) -> tuple[int, int]:
        """ returns F(k), F(k+1) """
        if k <= 1:
            return (k, 1)
        fh, fh1 = fib_pair(k>>1)
        fk = fh * ((fh1<<1) - fh)
        fk1 = fh1**2 + fh**2
        if k & 1:
            fk, fk1 = fk1, fk+fk1
        return fk, fk1
    return fib_pair(n)[0]
```

Figure 18 Fibonacci Fast Double Implementation

Results:

Below are the time results of the Fast Double method. This method is even faster than previous best Fast Matrix Multiplication method. This method is performed faster, because there are less multiplications and more bitwise operations, which are inherently more efficient.

Fast double																
Index:	501	631	794	1000	1259	1585	1995	2512	3162	3981	5012	6310	7943	10000	12589	15849
Time (ms):	0.0100	0.0069	0.0069	0.0079	0.0081	0.0408	0.0112	0.0126	0.0141	0.0191	0.0219	0.0281	0.0360	0.0520	0.0660	0.0980

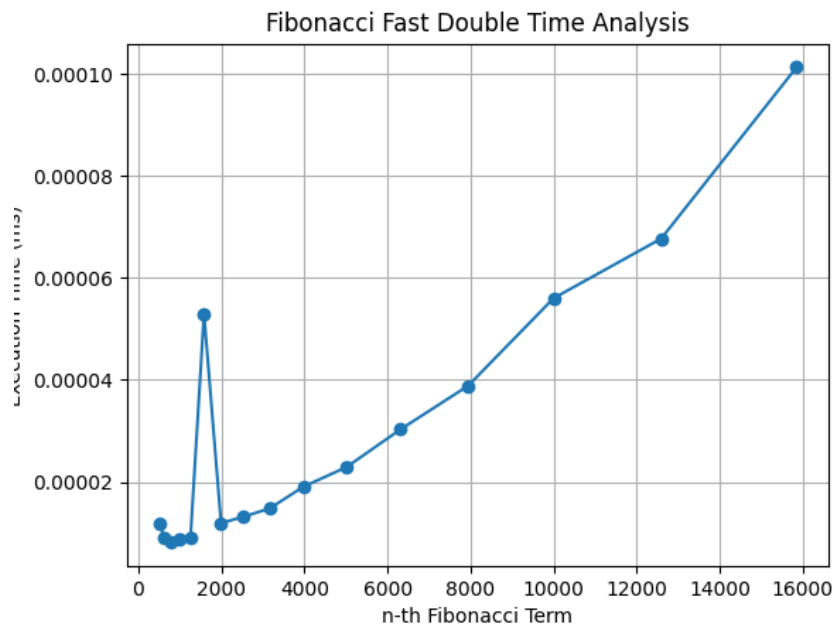


Figure 19 Fibonacci Fast Double Time Analysis

As in previous algorithm, k is reduced by a magnitude of 2 on each recursive call, so time complexity of this algorithm is $O(\log n)$. Similarly, creating local variables inside the `fib_pair` function also depends on the amount of recursive calls, so the space complexity is also $O(\log n)$.

Memoization:

This method proposes to store previously computed fibonacci numbers to avoid computing them again. This should come as a solution to the fundamental flaw of the textbook recursion method, which ends up computing the same fibonacci number multiple times.

Algorithm Description:

```
Fibonacci_memo(n):  
    If  $n \leq 1$ : Return  $n$   
    If  $n$  is already cached: Return cached result  
    result  $\leftarrow$  Fibonacci_memo( $n - 1$ ) + Fibonacci_memo( $n - 2$ )  
    Cache result for  $n$   
    Return result
```

Implementation:

The following implementation caches the previously computed results using the `@lru_cache` decorator and reads them from cache, if the value was already calculated.

```
@lru_cache(maxsize=None)
def fibonacci_memo(n: int) -> int:
    if n <= 1:
        return n
    return fibonacci_memo(n - 1) + fibonacci_memo(n - 2)
```

Figure 20 Fibonacci Memoization Implementation

Results:

Below is the table with the runtime for Memoization approach. Even though this approach is faster than the naive one, it is nowhere near as fast as Fast Matrix Multiplication or Fast double methods.

Memoization																
Index:	501	631	794	1000	1259	1585	1995	2512	3162	3981	5012	6310	7943	10000	12589	15849
Time (ms):	0.1712	0.0267	0.0362	0.0451	0.0551	0.0808	0.1001	0.1328	0.2410	0.2632	0.7238	0.5751	0.7610	1.0540	1.6110	2.2447

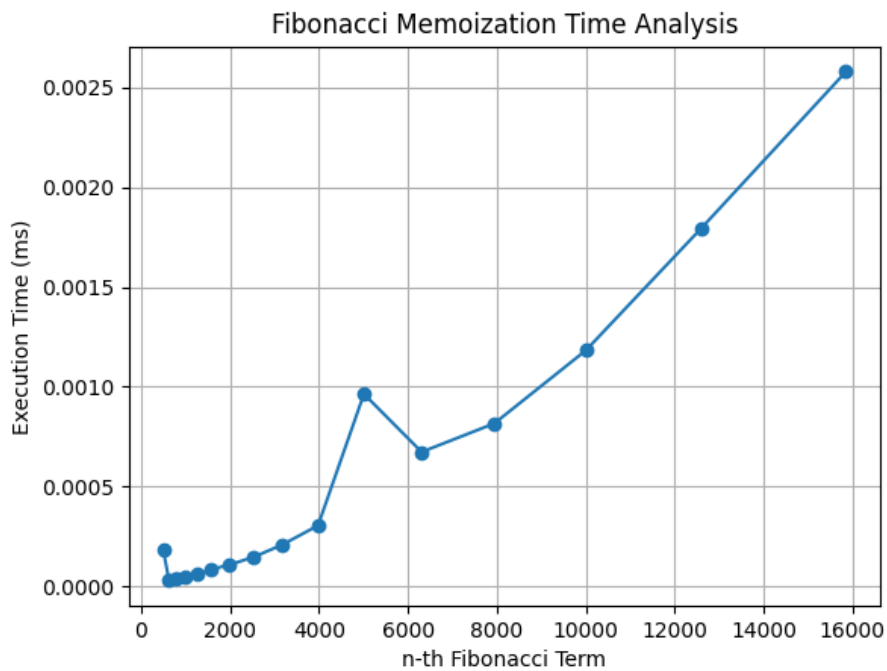


Figure 21 Fibonacci Memoization Time Analysis

From the graph and from the table it is evident that the bigger the fibonacci term, the more resources it consumes. Both time complexity and space complexity are asymptotically reaching linear complexity, so we can summarize it as a $O(n)$ approach.

CONCLUSION

Through Empirical Analysis, within this paper, four classes of methods have been tested in their efficiency at both their providing of accurate results, as well as at the time complexity required for their execution, to delimit the scopes within which each could be used, as well as possible improvements that could be further done to make them more feasible.

The Recursive method, being the easiest to write, but also the most difficult to execute with an exponential time complexity, can be used for smaller order numbers, such as numbers of order up to 30 with no additional strain on the computing machine and no need for testing of patience.

The Binet method, the easiest to execute with an almost constant time complexity, could be used when computing numbers of order up to 80, after the recursive method becomes unfeasible. However, its results are recommended to be verified depending on the language used, as there could rounding errors due to its formula that uses the Golden Ratio.

The Dynamic Programming and Matrix Multiplication Methods can be used to compute Fibonacci numbers further then the ones specified above, both of them presenting exact results and showing a linear complexity in their naivety that could be, with additional tricks and optimisations, reduced to logarithmic.

The Fast Matrix Multiplication approach avoids the inefficiency of naive recursion and is particularly useful for large values of n . It is faster than $O(n)$ iterative methods and maintains a low $O(\log n)$ space complexity if implemented iteratively. However, it requires matrix multiplication operations, making it slightly more complex than simple iterative approaches.

The Fast Double method significantly reduces redundant calculations compared to naive recursion. While the recursive implementation has $O(\log n)$ space complexity due to stack calls, an iterative version can reduce this to $O(1)$. This method is particularly effective for programming languages where recursion depth is not a concern and provides an efficient alternative to matrix exponentiation with fewer multiplications.

The Memoization optimizes the naive recursive Fibonacci function by storing previously computed values, preventing redundant calculations. While effective for moderate values of n , memoization suffers from recursion depth limits, making it less suitable for extremely large Fibonacci numbers compared to fast doubling or matrix exponentiation.